CS4475 Fall 2026
Skyye De La Riva
csdelariva@miners.utep.edu

Lab Report
Submitted on 10/15/26

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

To implement getprocs() as a system call, I added the entries needed to define getprocs() as a system call so I changed proc.h, syscall.h, and syscall.c. I then added the struct pstat in pstat.h defining pid, proc state, size, ppid, and name.

```
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       init
2       sleeping        20480   1       sh
3       running         16384   2       ps
```

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

I did the same thing for task 1 by setting up setpriority() and getpriority() as system calls in proc.h, syscall.h, syscall.c, user.h, sysproc.c, and usys.pl.

```
$ testpriority
Test: Initial process priority is 0
Test: Attempting to set priority to 10...
Test: Priority successfully updated to 10
Test: Attempting to set an invalid priority (50)...
Test: Correctly rejected invalid priority 50.
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

To calculate age, we first must implement that whenever a process is runnable, it calculates the readytime by subtracting the current uptime from the readytime.

```
$ ps
pid     state           size    ppid    name
1       sleeping        16384   0       0               0       init
2       sleeping        20480   0       0               1       sh
3       running         16384   0       0               2       ps
```

Task 4. (15 pts) Implement a priority-based scheduler.

The files I changed were param.h to implement the scheduling policies at compile time and proc.c to code the priority-based scheduler within the scheduler() function. From this I learned how xv6 can use different types of schedulers based on what we decide to use. I was confused about how to code a scheduler, so I was a bit lost. However, I looked at the scheduler that was already coded and formatted the priority-based scheduler code similar to the existing scheduler.

Task 5. (15 pts) Add aging to your priority-based scheduler.

For every RUNNABLE process, the scheduler calculates an effective priority using the formula: effective_priority = priority + ticks – readytime. Once a waiting process ages enough, the scheduler selects it for execution. This guarantees progress and fairness, ensuring that no process suffers from starvation. Some problems I ran into were accessing system clock ticks from kernel/proc.c initially caused compilation and runtime linker issues because ticks resides in kernel/trap.c so I just declared extern uint ticks; globally at the top of kernel/proc.c. I also had triggered a panic issue due to null pointer or unsafe lock handoffs within the scheduler's candidate-selection loop due to yield() not being correctly within the loop.

Task 6. (5 pts) Summary

From this lab I learned how to xv6 read processes and what kind of state and other values they can have. I also learned how xv6 can use different types of scheduler policies such as using priority-based scheduler to add aging and what kind of priority a process can have. Some obstacles I encountered were I wasn’t outputting the correct features, or some were missing entirely and my values in certain columns such as priority.